

Event Loop в JavaScript

1. Що таке Event Loop?

Event Loop (цикл подій) - це механізм, який дозволяє JavaScript виконувати неблокуючі операції, незважаючи на те, що JavaScript є однопотоковою мовою.

Основні компоненти

```
// Event Loop працює з кількома ключовими компонентами:  
// 1. Call Stack (стек викликів)  
// 2. Web APIs (у браузері) або C++ APIs (у Node.js)  
// 3. Callback Queue (черга колбеків)  
// 4. Microtask Queue (черга мікрозадач)  
  
// Приклад синхронного коду  
function main() {  
  console.log('1');  
  console.log('2');  
  console.log('3');  
}  
  
main();  
// Вивід: 1, 2, 3 (послідовно)
```

2. Call Stack

Принцип роботи стеку викликів

```
function multiply(a, b) {  
  return a * b;  
}  
  
function square(n) {  
  return multiply(n, n);  
}  
  
function printSquare(n) {  
  const result = square(n);  
  console.log(result);  
}  
  
printSquare(4);  
// Call Stack формується так:  
// 1. printSquare(4)  
// 2. square(4)
```

```
// 3. multiply(4, 4)
// 4. console.log(16)
```

3. Макрозадачі (Tasks)

setTimeout і setInterval

```
console.log('Початок');

setTimeout(() => {
  console.log('Таймер 1');
}, 0);

setTimeout(() => {
  console.log('Таймер 2');
}, 0);

console.log('Кінець');

// Вивід:
// Початок
// Кінець
// Таймер 1
// Таймер 2
```

4. Мікрозадачі (Microtasks)

Promise і queueMicrotask

```
console.log('Початок');

Promise.resolve().then(() => {
  console.log('Мікрозадача 1');
});

queueMicrotask(() => {
  console.log('Мікрозадача 2');
});

setTimeout(() => {
  console.log('Макрозадача');
}, 0);

console.log('Кінець');

// Вивід:
// Початок
// Кінець
```

```
// Мікрозадача 1
// Мікрозадача 2
// Макрозадача
```

5. Порядок виконання

Пріоритети задач

```
console.log('Скрипт почався');

setTimeout(() => {
  console.log('setTimeout 1');
}, 0);

Promise.resolve()
  .then(() => {
    console.log('Promise 1');
    setTimeout(() => {
      console.log('setTimeout 2');
    }, 0);
  })
  .then(() => {
    console.log('Promise 2');
  });

console.log('Скрипт закінчився');

// Вивід:
// Скрипт почався
// Скрипт закінчився
// Promise 1
// Promise 2
// setTimeout 1
// setTimeout 2
```

6. Практичні приклади

Обробка подій

```
button.addEventListener('click', () => {
  Promise.resolve().then(() => {
    console.log('Мікрозадача в обробнику події');
  });

  console.log('Обробник події');
});

// При кліку:
```

```
// Обробник події  
// Мікрозадача в обробнику події
```

Асинхронні операції

```
async function example() {  
  console.log('1');  
  
  setTimeout(() => {  
    console.log('2');  
  }, 0);  
  
  await Promise.resolve();  
  console.log('3');  
  
  new Promise(resolve => {  
    console.log('4');  
    resolve();  
  }).then(() => {  
    console.log('5');  
  });  
  
  console.log('6');  
}  
  
example();  
console.log('7');
```

// Вивід:
// 1
// 7
// 3
// 4
// 6
// 5
// 2

7. Node.js Event Loop

Фази циклу подій в Node.js

```
// 1. timers: setTimeout, setInterval  
setTimeout(() => console.log('timer'), 0);  
  
// 2. pending callbacks: I/O операції  
fs.readFile('file.txt', () => console.log('file read'));  
  
// 3. idle, prepare: внутрішнє використання
```

```
// 4. poll: отримання нових I/O подій

// 5. check: setImmediate
setImmediate(() => console.log('immediate'));

// 6. close callbacks: socket.on('close', ...)
```

8. Відладка та розуміння

Візуалізація Event Loop

```
console.log('1'); // Синхронний код

setTimeout(() => {
  console.log('2'); // Макрозадача
}, 0);

Promise.resolve()
  .then(() => {
    console.log('3'); // Мікрозадача
    setTimeout(() => {
      console.log('4'); // Нова макрозадача
    }, 0);
  })
  .then(() => {
    console.log('5'); // Мікрозадача
  });

console.log('6'); // Синхронний код

// Порядок виконання:
// 1. Синхронний код: 1, 6
// 2. Мікрозадачі: 3, 5
// 3. Макрозадачі: 2, 4
```

9. Найкращі практики

Оптимізація продуктивності

```
// Погано: блокування Event Loop
function heavyOperation() {
  for (let i = 0; i < 1000000000; i++) {
    // Важкі обчислення
  }
}

// Добре: розбиття на частини
```

```
function chunkedOperation(start = 0, end = 1000000000, chunk = 1000000) {  
  return new Promise(resolve => {  
    if (start >= end) {  
      resolve();  
      return;  
    }  
  
    setTimeout(() => {  
      for (let i = start; i < Math.min(start + chunk, end); i++) {  
        // Частина обчислень  
      }  
      chunkedOperation(start + chunk, end, chunk).then(resolve);  
    }, 0);  
  });  
}
```